

Tamara Hernández Romero | A01735955

26 de agosto de 2026

Objetivo: implementar regresión lineal usando solo numpy

```
#librerías necesarias
import numpy as np
```

```
#1 generar dataset sintético con parámetros especificados
np.random.seed(42)
n=150 #número observaciones (datos)
x= np.random.rand(n,1)*10 #genera 150 random numbers entre 0 y 10
e= np.random.randn(n,1)*2 #genera el ruido e con desviación 2 normal
y= 2*x + 3 + e #regresión
```

```
import matplotlib.pyplot as plt
```

```
#chechar e
print("media de e: ", np.mean(e))
print(e[:10])
print("desviación estandar de e: ", np.std(e))
```

```
media de e: 0.09371590348438322
[[-0.38472193]
 [ 0.60309468]
 [-0.06942354]
 [-2.33735608]
 [ 2.28564563]
 [ 1.50386607]
 [ 1.58206389]
 [-1.81877491]
 [ 2.80558862]
 [-2.80370213]]
desviación estandar de e: 2.056823244139644
```

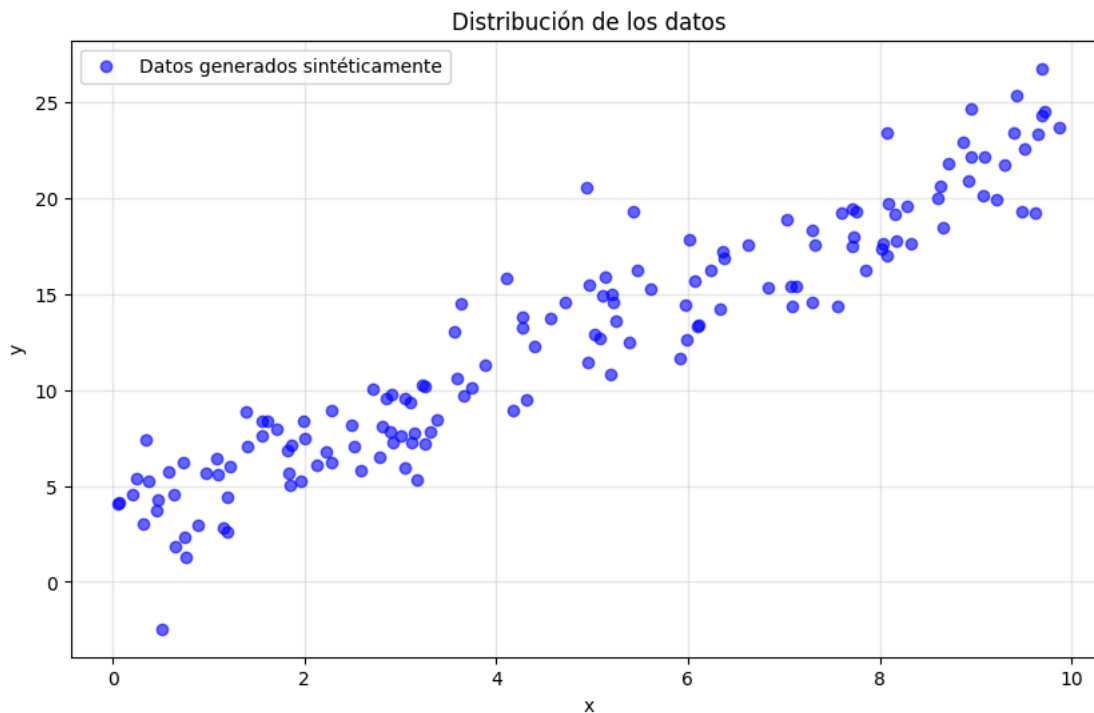
```
#exploración de los datos
print("largo de x", x.shape)
print("largo de y", y.shape)
media_x= np.mean(x)
media_y= np.mean(y)
#estadísticas
print("media de x: ", media_x)
print("media de y: ", media_y)
print("desviación estandar de x: ", np.std(x))
print("desviación estandar de y: ", np.std(y))

#visualización en gráfica de distribución de los datos
plt.figure(figsize=(10,6))
plt.plot(x,y,"o", alpha=0.6, color='blue', label='Datos generados sintéticamente')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Distribución de los datos')
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

```

largo de x (150, 1)
largo de y (150, 1)
media de x:  4.728871346995066
media de y:  12.551458597474515
desviación estandar de x:  2.9554068509950593
desviación estandar de y:  6.355344885214371

```



```

#2. calcular varianza
varianza_x=np.var(x)
varianza_e=np.var(e)
varianza_y=np.var(y)
print("varianza x: ", varianza_x)
print("varianza e: ", varianza_e)
print("varianza y: ", varianza_y)

```

```

varianza x:  8.734429654908531
varianza e:  4.23052185763313
varianza y:  40.390408610020465

```

```

#3. estimación de betas usando MCO
Sxx= np.sum((x-media_x)**2)
Sxy= np.sum((x-media_x)*(y-media_y)) #covarianza
print("Sxx:",Sxx)
pendiente_B1= (Sxy)/(Sxx)
intercepción_B0= media_y- pendiente_B1*media_x
print("pendiente B1: ", pendiente_B1) #real es 2
print("intercepción B0: ", intercepción_B0) #real es 3

```

```

Sxx: 1310.1644482362797
pendiente B1:  2.034981337678597
intercepción B0:  2.928293658056507

```

```

#4. calcular R^2 y MSE

```

```

#prediccion
y_pred=pendiente_B1*x+intercepción_B0
r2=(np.sum((y_pred-media_y)**2))/(np.sum((y-media_y)**2))
mse=np.mean((y-y_pred)**2)

print("R^2: ", r2)
print("MSE: ", mse)

```

```

R^2:  0.8955238698043709
MSE:  4.219833588595164

```

```
#5. visualización resultados
plt.figure(figsize=(10,6))
plt.plot(x,y,"o", alpha=0.6, color='blue', label='Datos generados sintéticamente')
plt.plot(x, y_pred, color='red', linewidth=2, label='Línea de regresión (MCO)')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Visualización de resultados')
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

